Documentación Técnica: Proyecto CarLog

Desarrollo de aplicaciones web - Javier Ramírez Serrano

Abril 2026

Índice

1. Análisis General del Backend **2**
1.1. Stack Tecnológico del Servidor 2

2. Arquitectura en Capas (Layered Architecture) **2**
2.1. Capa de Controladores (API Layer) 2
2.2. Capa de Servicios (Business Logic) 2
2.3. Capa de Persistencia 2

3. Sistema de Seguridad y Autenticación **3**
3.1. Flujo de Autenticación Stateless 3
3.2. Seguridad en el Cliente (Angular Guards) 3

4. Entidades de Dominio **3**

5. Despliegue y DevOps **3**

6. Sistema de Comunicación en Tiempo Real (WebSockets) **4**
6.1. Infraestructura y Mensajería (Backend) 4
6.2. Reactividad y Event Bus (Frontend) 4
6.3. Casos de Uso Críticos 5

7. Características de Diseño Avanzado **5**

8. Manejo Centralizado de Errores y Feedback Visual **5**
8.1. Backend: Global Exception Handler 5
8.2. Frontend: Interceptores y SweetAlert2 6

1 Análisis General del Backend

El ecosistema backend de CarLog está construido sobre un stack de última generación, priorizando la seguridad stateless y la escalabilidad multi-taller.

1.1. Stack Tecnológico del Servidor

- **Framework:** Spring Boot 4.0.1.
- **Lenguaje:** Java 21 (LTS).
- **Persistencia:** Spring Data JPA con MySQL 8.0.
- **Seguridad:** Spring Security con JWT (JSON Web Tokens).
- **Infraestructura:** Docker & Docker Compose.

2 Arquitectura en Capas (Layered Architecture)

El sistema sigue un patrón de diseño que separa la lógica de presentación de la lógica de negocio pura.

2.1. Capa de Controladores (API Layer)

Los controladores actúan como orquestadores de las peticiones REST. Implementan validaciones granulares y transforman entidades a DTOs.

- `VehicleController`: CRUD y flujos de entrada/salida.
- `UserController`: Gestión de clientes y empleados.
- `WorkOrderController`: Gestión de órdenes y líneas de servicio.
- `WorkshopController`: Administración de empleados y perfil del taller.

2.2. Capa de Servicios (Business Logic)

Es el corazón del sistema, donde se aplican las reglas de negocio y se gestiona la transaccionalidad mediante la anotación `@Transactional`.

- **Cálculo Financiero:** El `WorkOrderService` automatiza el cálculo de importes totales, procesando de forma dinámica las líneas de trabajo para desglosar la base imponible, cuotas de IVA y descuentos aplicados.
- **Gestión de Archivos:** Los servicios reciben archivos binarios a través de objetos `MultipartFile`. El sistema procesa estos flujos de datos para su persistencia física en el sistema de archivos del servidor (directorio `/uploads`), almacenando únicamente la referencia o metadatos necesarios en la base de datos MySQL.

2.3. Capa de Persistencia

Uso de repositorios JPA para la abstracción de consultas SQL. El sistema está diseñado para aislamiento de datos entre talleres.

3 Sistema de Seguridad y Autenticación

CarLog implementa una seguridad de nivel empresarial basada en tokens JWT.

3.1. Flujo de Autenticación Stateless

A diferencia de los sistemas tradicionales, el JWT no se envía en el cuerpo de la respuesta, sino que se inyecta en una **Cookie HTTP-only** para mitigar riesgos de seguridad.

Listing 1: Configuración del Filtro de Seguridad

```
// JwtAuthenticationFilter.java
// Intercepta peticiones y extrae el JWT de la cookie
String token = getJwtFromCookie(request);
if (jwtService.isTokenValid(token, userDetails)) {
    SecurityContextHolder.getContext().setAuthentication(authToken);
}
```

3.2. Seguridad en el Cliente (Angular Guards)

En la capa de presentación (Frontend), el control de acceso y la protección de rutas se gestiona mediante **Guards de Angular**, asegurando que los usuarios solo accedan a las vistas permitidas según su estado de sesión y rol de negocio:

- `loginGuard`**:** Redirige a los usuarios que ya poseen una sesión activa directamente al dashboard, evitando accesos redundantes a la pantalla de autenticación.
- `authguardGuard`**:** Actúa como barrera principal protegiendo todas las rutas internas de la aplicación, exigiendo que exista un token JWT válido en el *Storage* (Local o Session).
- `tallerGuard`**:** Implementa lógica de negocio específica. Verifica que un usuario con rol directivo tenga un taller registrado antes de permitirle el acceso a las herramientas de gestión del taller, evitando inconsistencias de datos.
- `altaTallerGuard`**:** Controla el acceso al formulario de creación de talleres, bloqueando la ruta para aquellos usuarios que ya poseen un taller registrado, previniendo duplicidades.

4 Entidades de Dominio

Entidad	Responsabilidad Crítica
`User`	Gestión RBAC (MANAGER, MECHANIC, CLIENT, DIY).
`Vehicle`	Almacena especificaciones técnicas e historial de taller.
`WorkOrder`	Cabecera de reparación con estados: PENDING, IN_PROGRESS, COMPLETED.
`Workshop`	Entidad multi-tenant que aísla empleados y vehículos.

5 Despliegue y DevOps

El proyecto incluye configuración nativa para entornos de contenedores, facilitando el despliegue en entornos de staging y producción.

Listing 2: Estructura de Docker Compose

```yaml
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: carlog_db
  backend:
    build: .
    ports:
      - "8081:8081"
    depends_on:
      - mysql
```

6 Sistema de Comunicación en Tiempo Real (WebSockets)

Para garantizar una experiencia de usuario fluida y reactiva, CarLog implementa un sistema de notificaciones asíncronas basado en el protocolo **STOMP** (*Simple Text Oriented Messaging Protocol*) sobre **WebSockets**.

6.1. Infraestructura y Mensajería (Backend)

El servidor actúa como un broker de mensajes utilizando `Spring WebSocket`. La comunicación se gestiona mediante el componente `SimpMessagingTemplate`, que permite el envío dirigido de mensajes a tópicos específicos.

- **Endpoint de Conexión:** `/ws-carlog/**` (Configurado como público en `SecurityConfig` para permitir el *handshake* inicial).
- **Estructura de Tópicos:** Se utiliza una arquitectura de tópicos privados por usuario: `/topic/notificaciones/{dni}`.
- **Protocolo de Datos:** Los mensajes se encapsulan en el objeto `NotificationDTO`, compuesto por: `tipo`, `título` y `mensaje`.

Listing 3: Ejemplos de emisión de notificaciones en Spring Boot

```java
// 1. Flujo de Invitaciones Laborales
messagingTemplate.convertAndSend("/topic/notificaciones/" + employeeDni, notif);

// 2. Flujo de Aceptaci n de Propuesta
messagingTemplate.convertAndSend("/topic/notificaciones/" + managerDni, alert);

// 3. Solicitud de Ingreso de Veh culo
messagingTemplate.convertAndSend("/topic/notificaciones/" + ownerDni, notif);
```

6.2. Reactividad y Event Bus (Frontend)

En el cliente, la aplicación Angular utiliza la librería `RxStomp` para mantener la conexión activa. Para desacoplar la recepción del mensaje de la lógica de los componentes, se implementa un **Event Bus Reactivo** mediante el servicio `NotificationBusService`.

- **Tipos de Eventos (`AppEventType`):** `RELOAD_VEHICLES`, `RELOAD_EMPLOYEES`, `NEW_INVITE`.
- **Distribución de Señales:** Al recibir un mensaje por el socket, el `DashboardLayout` captura la notificación y emite un evento a través del bus, activando recargas automáticas de datos en los componentes suscritos sin necesidad de refrescar la página.

Listing 4: Suscripcion y emision de eventos en Angular

```
this.rxStomp.watch(`/topic/notificaciones/${miDni}`).subscribe((message) => {
    const notification = JSON.parse(message.body);

    switch (notification.type) {
        case 'VEHICLE_REQUEST':
            this.mostrarToast(notification);
            this.notificationBus.emit(AppEventType.RELOAD_VEHICLES);
            break;
        case 'FIRE':
            this.authService.logout(); // Desconexion inmediata por baja laboral
            break;
    }
});
```

6.3. Casos de Uso Críticos

1. **Gestión de Personal:** Permite a los Managers recibir confirmaciones de contratación al instante y a los empleados ser notificados de su cese laboral (`FIRE`), forzando un cierre de sesión seguro.

2. **Validación de Dueño:** Cuando un taller solicita el ingreso de un vehículo, el propietario recibe una alerta instantánea para autorizar la operación desde su panel de "Mis Vehículos", agilizando el flujo de entrada al taller.

7 Características de Diseño Avanzado

- **Separación DTO/Entity:** Evita la exposición de datos sensibles y optimiza las respuestas JSON.

- **WebSockets:** Integración con STOMP para notificaciones reactivas (Invitaciones, alertas de ingreso).

- **Gestión de Archivos:** Persistencia física en la carpeta `/uploads` con lógica de limpieza automática de archivos huérfanos.

8 Manejo Centralizado de Errores y Feedback Visual

El sistema implementa una arquitectura robusta para la captura, procesamiento y presentación de errores, abarcando desde excepciones en la base de datos hasta la interfaz del usuario, garantizando la estabilidad y una experiencia de usuario (UX) óptima.

8.1. Backend: Global Exception Handler

En la capa de Spring Boot, se emplea la anotación `@RestControllerAdvice` acoplada a la clase `GlobalExceptionHandler` para centralizar el manejo de excepciones de toda la API. En lugar de devolver *stack traces* nativos al cliente (práctica que expone vulnerabilidades y rompe el cliente), este interceptor captura excepciones específicas de negocio (como `VehicleNotFoundException`, `UserNotFoundException` o `VehicleOcuppiedException`) y construye una respuesta HTTP estandarizada. Este DTO de error incluye siempre el mensaje validado de la excepción, el código de estado HTTP adecuado (404, 409, 500) y un *timestamp*.

8.2. Frontend: Interceptores y SweetAlert2

En el lado del cliente (Angular), la asimetría de errores se gestiona mediante un Interceptor HTTP (`error-interceptor.ts`). Este servicio global captura las respuestas de error estandarizadas emitidas por el `GlobalExceptionHandler`. Una vez interceptado el error, el sistema previene el bloqueo de la aplicación y extrae el mensaje limpio para proporcionar *feedback* visual asíncrono utilizando la librería **SweetAlert2**. Estas alertas modales o notificaciones tipo *toast* están diseñadas de forma coherente con el *Dark Theme* de la aplicación (usando colores hexadecimales como `#212529`), aportando información clara al usuario final (por ejemplo, .El taller ya tiene acceso a tu vehículo.º Çredenciales incorrectas") sin interrumpir su flujo de navegación.